Grammars, Parsing & ASTs

Liting Zhang

University of West Florida

Big Picture: Where Are We in the Compiler Pipeline?

- So far (Week 2): lexical structure
  - Raw source $\rightarrow$ characters $\rightarrow$ tokens
  - Tokens: keywords, identifiers, literals, operators, separators, comments
  - Implemented with regular expressions and a tiny JavaScript lexer
- This week (Week 3): syntax
  - How do we describe valid sequences of tokens?
  - How do we turn a token stream into a tree structure?
- Pipeline view:

$$\text{Source} \rightarrow \text{Lexer} \rightarrow \text{Parser} \rightarrow \text{AST} \rightarrow \text{Later passes}$$

Week 3 Plan

- Context-free grammars (CFGs)
- BNF/EBNF notation
- Derivations and parse trees
- Ambiguity, precedence, associativity
- Abstract syntax trees (ASTs) vs parse trees
- Recursive descent parsing (top-down, hand-written)
- JavaScript demo: expression parser that builds an AST
- Java/C++ grammar snippets, parser generators (ANTLR)

Why Do We Need Grammars?

- Natural language analogy:
  - English has nouns, verbs, sentences, and rules for combining them.
  - Programming languages also have rules: "an if statement looks like this. . ."
- We want a precise, formal way to say:
  - which token sequences are valid programs
  - how they are structured as trees
- Grammars:
  - are the standard formalism for language syntax
  - are used by parser generators and hand-written parsers

Context-Free Grammars (CFGs): Informal Idea

A context-free grammar describes valid strings with production rules. Core ingredients:

- Terminals:
  - things that actually appear in the input token stream
  - examples: `if`, `else`, `+`, `(`, `)`, `IDENT`, `NUM`
  - in practice, these often correspond to lexer token types
- Nonterminals:
  - names for syntactic categories (abstract "slots" in the structure)
  - examples: `Expr`, `Stmt`, `Block`, `Type`
  - do not appear literally in the source code
- Start symbol:
  - the nonterminal that represents a complete valid input
  - often called `Program` or `CompilationUnit`
- Productions: how a nonterminal expands into terminals and other nonterminals
  - `Expr -> Expr "+" Term`
  - `Stmt -> "if" "(" Expr ")" Stmt`

Terminals vs Nonterminals

Consider the source fragment:

```
1  42 + x * (y + 3)
```

After lexical analysis (Week 2), we might get tokens:

```
NUMBER(42)   PLUS("+")   IDENT(x)   STAR("*")
LPAREN("(")  IDENT(y)    PLUS("+")  NUMBER(3)  RPAREN(")")
```

- Terminals: token types such as `NUMBER`, `PLUS`, `IDENT`, `STAR`, `LPAREN`, `RPAREN`
- Nonterminals: categories we use in the grammar, for example
  - `Expr` for "any expression"
  - `Term` for "multiplication/division group"
  - `Factor` for "single number, variable, or parenthesized expression"
- Intuition: terminals are the leaves of the parse tree, nonterminals are the internal nodes that organize them.

A Tiny Arithmetic Expression Grammar

Terminals (coming from the lexer):

- `NUM`, `+`, `*`, `(`, `)`

Nonterminals:

- `Expr`, `Term`, `Factor`

Grammar:

$$\begin{aligned} \text{Expr} &\rightarrow \text{Expr} + \text{Term} \mid \text{Term} \\ \text{Term} &\rightarrow \text{Term} * \text{Factor} \mid \text{Factor} \\ \text{Factor} &\rightarrow (\text{Expr}) \mid \text{NUM} \end{aligned}$$

This grammar generates strings like:

- `NUM`
- `NUM + NUM`
- `NUM * NUM + NUM`
- `(NUM + NUM) * NUM`

BNF Notation

BNF = Backus–Naur Form

Common conventions:

- Nonterminals: capitalized or in angle brackets, e.g. `<expr>`
- Terminals: literal symbols or tokens, e.g. `"+"`, `"if"`
- Productions: $< expr >::=< expr >$ " + " $< term > | < term >$

Arithmetic example in BNF:

```
<expr>   ::= <expr> "+" <term> | <term>
<term>   ::= <term> "*" <factor> | <factor>
<factor> ::= "(" <expr> ")" | NUM
```

BNF is widely used in language specifications (for example, early C and Pascal).

EBNF: Extended BNF

EBNF adds shorthand:

- `A*` = zero or more `A`
- `A+` = one or more `A`
- `[A]` = optional `A`

Example: a comma-separated list of identifiers

```
<id-list> ::= IDENT ("," IDENT)*
```

Expression example:

```
<expr>   ::= <term> ("+" <term>)*
<term>   ::= <factor> ("*" <factor>)*
<factor> ::= "(" <expr> ")" | NUM
```

EBNF is easier to read and closer to how we think about repetition and options.

Derivations: How Grammars Generate Strings

- A derivation starts at the start symbol and repeatedly applies productions.
- Example: derive `NUM + NUM * NUM`

Using the arithmetic grammar:

$$\begin{aligned}
\text{Expr} &\Rightarrow \text{Expr} + \text{Term} \\
&\Rightarrow \text{Term} + \text{Term} \\
&\Rightarrow \text{Factor} + \text{Term} \\
&\Rightarrow \text{NUM} + \text{Term} \\
&\Rightarrow \text{NUM} + \text{Term} * \text{Factor} \\
&\Rightarrow \text{NUM} + \text{Factor} * \text{Factor} \\
&\Rightarrow \text{NUM} + \text{NUM} * \text{NUM}
\end{aligned}$$

Different derivation orders (leftmost vs rightmost) give the same final string, but may correspond to different parse trees for ambiguous grammars.

Parse Trees

A parse tree (also called a concrete syntax tree):

- reflects the structure of a derivation
- has nonterminals as internal nodes
- has terminals (tokens) as leaves

For `NUM + NUM * NUM` with our grammar:

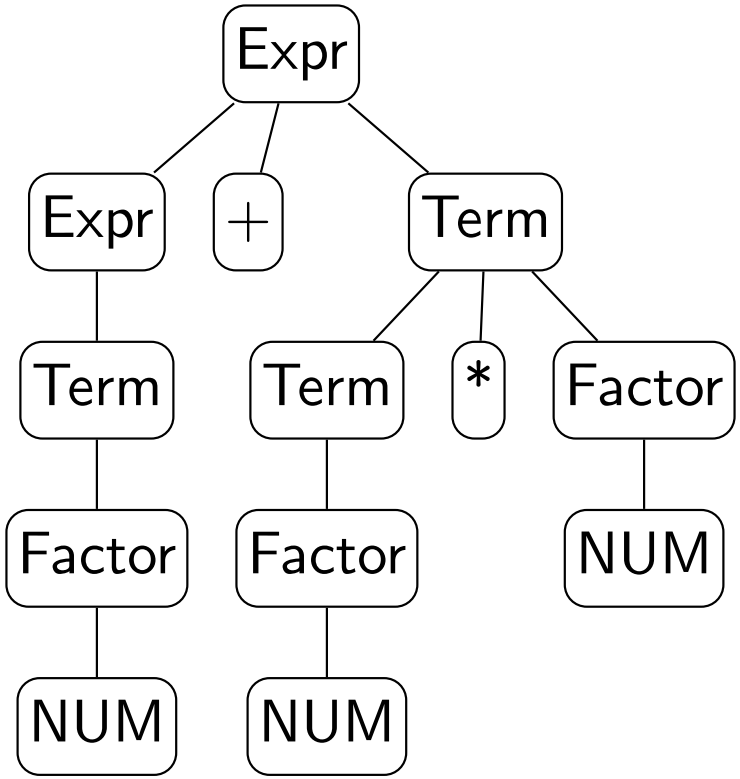
- Root: `Expr`
- Children: `Expr`, `+`, `Term`
- `Expr` child: `Term`
- `Term` child: `Factor`
- and so on

Parse Tree for NUM + NUM * NUM

- Root nonterminal: `Expr`
- Grammar enforces that * binds tighter than +

Grammar:

```
<Expr>   ::= <Expr> "+" <Term> | <Term>
<Term>   ::= <Term> "*" <Factor> | <Factor>
<Factor> ::= "(" <Expr> ")" | NUM
```



Ambiguity in Grammars

A grammar is ambiguous if at least one string has two different parse trees.

Example of an ambiguous expression grammar:

```
Expr ::= Expr "+" Expr
       | Expr "*" Expr
       | NUM
```

- The string `NUM + NUM * NUM` can parse as:
  - `(NUM + NUM) * NUM`
  - `NUM + (NUM * NUM)`
- Arithmetic meaning is usually * before +.

Real languages avoid ambiguity (or define disambiguation rules) to make parsing deterministic and semantics clear.

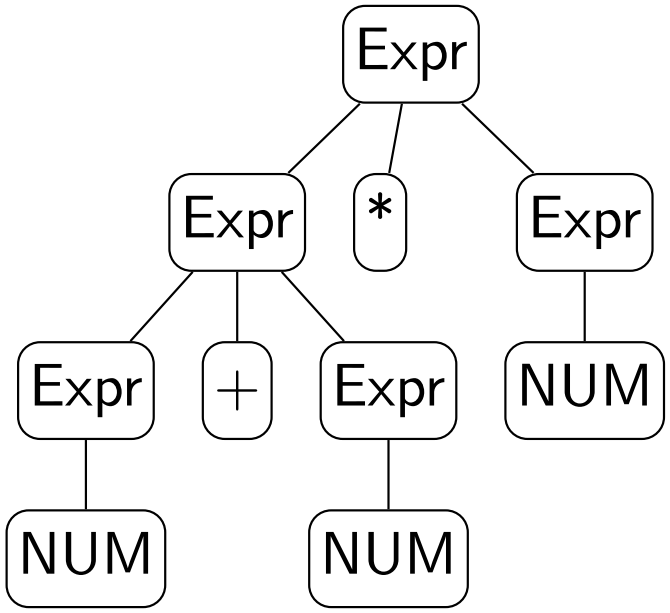
Ambiguous Parse Trees for NUM + NUM * NUM

We use the ambiguous grammar:

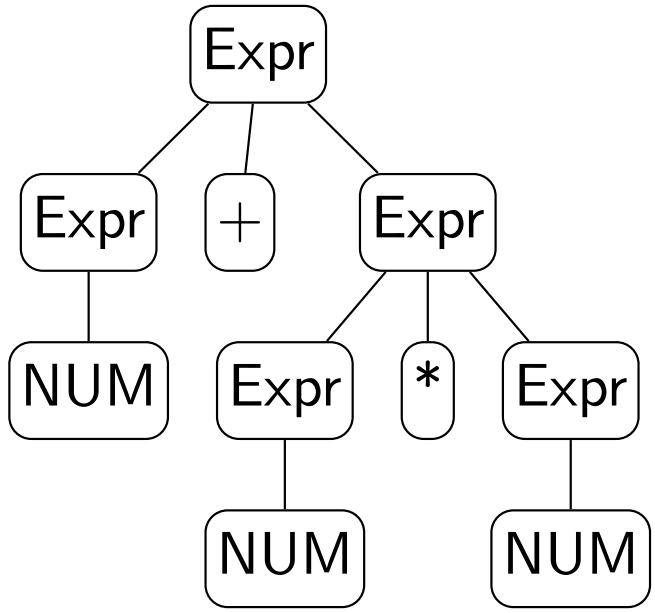
```
Expr ::= Expr "+" Expr
       | Expr "*" Expr
       | NUM
```

For the string NUM + NUM * NUM, there are two parse trees:

`(NUM + NUM) * NUM`



`NUM + (NUM * NUM)`



Ambiguity: The Dangling Else Problem

Classic example in many language grammars:

```
Stmt ::= "if" "(" Expr ")" Stmt
       | "if" "(" Expr ")" Stmt "else" Stmt
       | ...
```

The string:

```
if (c1)
  if (c2)
    s1;
  else
    s2;
```

Two possible parses:

- `else` matches the inner `if`
- `else` matches the outer `if`

Most languages (C, Java, JavaScript) use the rule:

An `else` *is matched with the nearest preceding unmatched* `if`*.*

Fixing Precedence and Associativity

We can encode precedence and associativity in the grammar.

Precedence (multiplication tighter than addition):

```
Expr   ::= Term ("+" Term)*
Term   ::= Factor ("*" Factor)*
Factor ::= "(" Expr ")" | NUM
```

Associativity example (left-associative +):

- Expr $\rightarrow$ Expr "+" Term | Term
- or with EBNF: `Expr -> Term ("+" Term)*`
- This forces `a + b + c` to parse as `(a + b) + c`.

Designing grammars is partly math and partly language design.

Java and C++ Grammar Snippets

- Real languages have large grammars (hundreds of rules).
- Example, simplified from a Java expression grammar:

```
1  Expression
2    ::= AssignmentExpression
3
4  AssignmentExpression
5    ::= ConditionalExpression
6     |  LeftHandSideExpression AssignmentOperator Expression
7
8  ConditionalExpression
9    ::= LogicalOrExpression
10    |  LogicalOrExpression "?" Expression ":" ConditionalExpression
```

- C++ has an even more complex grammar because of templates, operator overloading, and other features.
- Takeaway: the same CFG ideas scale up to industrial languages.

Summary So Far

- Tokens from the lexer are organized by a parser using a grammar.
- CFGs describe syntax via terminals, nonterminals, productions, and a start symbol.
- BNF and EBNF are notations for writing grammars.
- Derivations and parse trees show how strings are generated.
- Ambiguity is problematic for compilers; we often fix it using precedence and associativity.

From Parse Trees to ASTs

Parse tree:

- very close to the grammar
- includes all parentheses, keywords, and punctuation
- often quite bushy

Abstract syntax tree (AST):

- more compact, semantics-oriented structure
- drops syntax sugar and unnecessary nodes
- children represent meaningful subexpressions, not just grammar artifacts

AST vs Parse Tree: Visual Comparison

- Parse tree closely mirrors the grammar:
  - nodes such as `Expr`, `Term`, `Factor`
  - parentheses appear as their own leaves
- AST:
  - internal nodes represent operations, such as `BinaryExpr`, `UnaryExpr`, `CallExpr`
  - leaves represent identifiers and literals, such as `Identifier`, `Number`, `String`

In implementation:

- AST nodes are usually classes or structured objects.
- Later phases (type checking, interpretation, code generation) use the AST.

AST Node Shapes for a Tiny Expression Language

Suppose we support:

- numbers
- binary operators + and *
- parentheses

We might define AST node "types" as JavaScript objects:

```
{ type: "NumberLiteral", value: 42 }
```

```
{
  type: "BinaryExpr",
  op: "+",
  left:  { /* another AST node */ },
  right: { /* another AST node */ }
}
```

Later, an interpreter can evaluate:

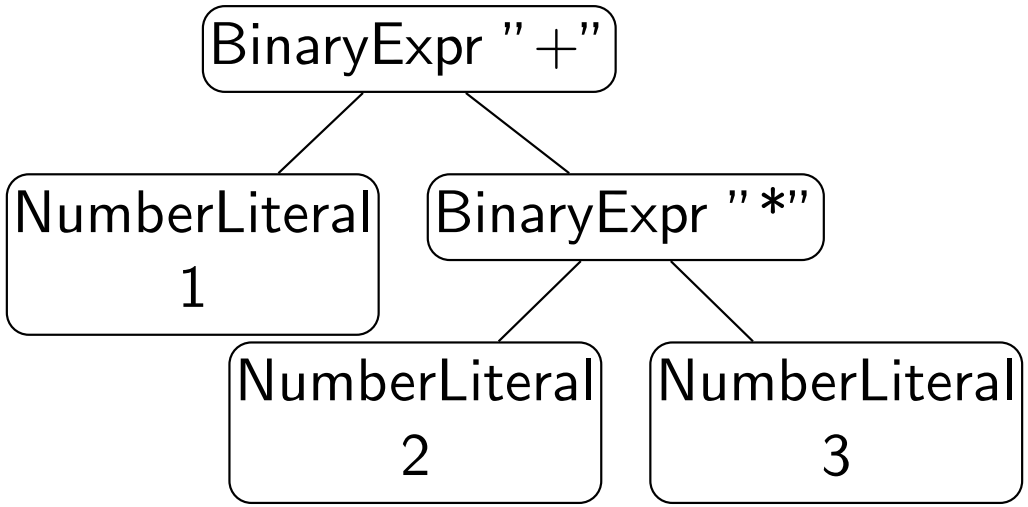
- `NumberLiteral` by returning its value
- `BinaryExpr` by recursively evaluating `left` and `right`, then applying `op`

Example AST for 1 + 2 * 3

A possible AST (as JavaScript-style objects):

```
{
  type: "BinaryExpr",
  op: "+",
  left:  { type: "NumberLiteral", value: 1 },
  right: {
    type: "BinaryExpr",
    op: "*",
    left:  { type: "NumberLiteral", value: 2 },
    right: { type: "NumberLiteral", value: 3 }
  }
}
```

Visual tree:



Industry angle: how TypeScript / JavaScript tools parse code

Modern tooling for TypeScript and JavaScript (editors and CLIs) uses parsers:

- language servers (for example VS Code) provide:
  - syntax highlighting
  - go-to-definition, rename, code completion
- analysis tools:
  - ESLint, TypeScript compiler, static analyzers
- code transformation tools:
  - Babel, swc, TypeScript compiler emitters

All of these have a pipeline very similar to our toy compiler:

$$\text{Source} \rightarrow \text{Lexer} \rightarrow \text{Parser} \rightarrow \text{AST} \rightarrow \text{Analysis / Transformations}$$

Top-down and bottom-up parsing

Two big families of parsing strategies:

Top-down parsing:

- start from the start symbol (for example `Expr`)
- predict which productions to use, based on the next tokens
- build the tree from the root down toward the leaves
- examples:
  - hand-written recursive descent (what we are doing)
  - LL (Left-to-right, Leftmost derivation) parsers generated by tools

Bottom-up parsing:

- start from the input tokens
- repeatedly combine tokens and partial subtrees into larger phrases
- build the tree from the leaves up toward the root
- examples:
  - LR (left-to-right, rightmost-derivation-in-reverse), LALR, GLR parsers
  - traditional parser generators such as YACC, Bison

Top-down vs bottom-up in real parsers

In practice:

- many modern language implementations use:
  - hand-written recursive descent (top-down), or
  - parser generators that produce LL-style top-down parsers
- classic C and C++ compilers have often used:
  - LALR bottom-up parsers generated by YACC or Bison

Reasons to choose one or the other include:

- control over error messages and recovery
- convenience of writing and maintaining the grammar
- performance and the complexity of the language syntax

For this course, recursive descent is a good model: the code is small enough to read on a slide, and it mirrors the grammar directly.

Recursive Descent Parsing: Idea

- Top-down parser: start from the start symbol and predict what comes next.
- For each nonterminal in the grammar, write a function that:
  - consumes tokens from the input
  - returns an AST node
  - throws an error if the syntax is invalid
- This approach is natural to implement by hand for smaller languages or subsets.

Example mapping:

- nonterminal `Expr` corresponds to function `parseExpr`
- nonterminal `Term` corresponds to function `parseTerm`
- nonterminal `Factor` corresponds to function `parseFactor`

Designing a grammar for expressions

Suppose we want a tiny expression language with:

- numbers, +, −, *, /
- parentheses for grouping
- usual math rules:
  - * and / bind tighter than + and −
  - +, −, *, / are left-associative

A naive grammar might be:

```
Expr ::= Expr "+" Expr
       | Expr "*" Expr
       | NUM
```

Problems:

- ambiguous: `NUM + NUM * NUM` has two parse trees
- no explicit precedence or associativity

Our design strategy:

- introduce layers that match precedence: `Expr` for +- and `Term` for */
- use repetition patterns (or left recursion) to encode associativity
- let `Factor` handle base cases and parentheses

From operator table to Expr / Term / Factor

Start from the operator table:

- highest: parentheses, numbers
- then: `*`, `/`
- then: `+`, `-`

Design a layered grammar:

```
Expr   ::= Term (("+"|"-") Term)*
Term   ::= Factor (("*"|"/") Factor)*
Factor ::= "(" Expr ")" | NUM
```

Why this matches our design goals:

- precedence:
  - `Expr` is built out of `Terms`, so `*` and `/` are resolved before `+` and `-`.
- associativity:
  - `("+" Term)*` means a left-fold of `+` over `Terms`, so `a + b + c` parses as `(a + b) + c`.
- parentheses:
  - `Factor ::= "(" Expr ")"` lets any full expression appear as a single factor, overriding normal precedence.

Grammar for Our Recursive Descent Parser

We use a precedence-aware grammar that is easy to parse top-down:

```
Expr   ::= Term (("+"|"-") Term)*
Term   ::= Factor (("*"|"/") Factor)*
Factor ::= "(" Expr ")" | NUM
```

Tokens (from a Week 2 style lexer):

- `NUMBER` tokens with a numeric value
- symbol tokens for `+`, `-`, `*`, `/`, `(` , `)`
- possibly an `EOF` token to mark the end

Each nonterminal becomes a JavaScript function that:

- looks at the current token
- decides which production to use
- builds AST nodes

Token Stream Helper in JavaScript

Assume we already have a lexer that produces an array of tokens like:

- `{ type: "NUMBER", value: 42 }`
- `{ type: "PLUS",   lexeme: "+" }`

```
1  class Parser {
2    constructor(tokens) {
3      this.tokens = tokens;
4      this.pos = 0;
5    }
6    current() {
7      return this.tokens[this.pos];
8    }
9    match(type) {
10     const tok = this.current();
11     if (tok && tok.type === type){
12       this.pos++;
13       return tok;
14     }
15     return null;
16   }
17   expect(type, message) {
18     const tok = this.match(type);
19     if (!tok) {
20       throw new Error(
21         message + " at token "+
22         JSON.stringify(this.current())
23       );
24     }
25     return tok;
26   }
27 }
```

Parsing Factors

Grammar:

```
Factor ::= "(" Expr ")" | NUM
```

```
1  parseFactor() {
2    const tok = this.current();
3    // "(" Expr ")"
4    if (tok && tok.type === TokenType.LPAREN) {
5      this.match(TokenType.LPAREN);
6      const expr = this.parseExpr();
7      this.expect(TokenType.RPAREN, "Expected ')'");
8      // We do not keep parentheses as separate AST nodes
9      return expr;
10   }
11   // NUM
12   if (tok && tok.type === TokenType.NUMBER) {
13     this.match(TokenType.NUMBER);
14     return {
15       type: "NumberLiteral",
16       value: tok.value
17     };
18   }
19   throw new Error(
20     "Expected number or '(' at token " + JSON.stringify(tok)
21   );
22 }
```

Parsing Terms with Left-Associative Loops

Grammar: $Term ::= Factor((" * "|" / ")Factor)*$

- parse a `Factor`
- while the next token is `"*"` or `"/"`, consume it and another `Factor`
- each time, build a `BinaryExpr` whose `left` child is the previous node

```
1  parseTerm() {
2    let node = this.parseFactor();
3    while (true) {
4      const tok = this.current();
5      if(!tok || (tok.type !== TokenType.STAR && tok.type !== TokenType.SLASH)) {
6        break;
7      }
8      this.pos++;// consume * or /
9      const right = this.parseFactor();
10     node = {
11       type: "BinaryExpr",
12       op: tok.lexeme, // "*" or "/"
13       left: node,
14       right: right
15     };
16   }
17   return node;
18 }
```

Parsing Expressions (Plus and Minus)

Grammar: $Expr ::= Term((" + " | " - ")\, Term)*$

Implementation (same pattern as `parseTerm`):

```
1  parseExpr() {
2    let node = this.parseTerm();
3    while (true) {
4      const tok = this.current();
5      if(!tok || (tok.type !== TokenType.PLUS && tok.type !== TokenType.MINUS)) {
6        break;
7      }
8      this.pos++;// consume + or -
9      const right = this.parseTerm();
10     node = {
11       type: "BinaryExpr",
12       op: tok.lexeme, // "+" or "-"
13       left: node,
14       right: right
15     };
16   }
17   return node;
18 }
```

Putting It Together: Top-Level Parse Function

We want to parse a whole expression and ensure we consumed all tokens.

```
parse() {
  const ast = this.parseExpr();
  const tok = this.current();
  if (tok && tok.type !== TokenType.EOF) {
    throw new Error(
      "Unexpected extra input at token " + JSON.stringify(tok)
    );
  }
  return ast;
}
```

Usage example: node expr_parser.js

```
const tokens = lex("1 + 2 * 3");   // from a Week 2 style lexer
tokens.push({ type: "EOF" });
const parser = new Parser(tokens);
const ast = parser.parse();
console.log(JSON.stringify(ast, null, 2));
```

What Does the AST Look Like for 1 + 2 * 3?

For input `1 + 2 * 3`, the AST should encode precedence:

```
{
  "type": "BinaryExpr",
  "op": "+",
  "left": { "type": "NumberLiteral", "value": 1 },
  "right": {
    "type": "BinaryExpr",
    "op": "*",
    "left":  { "type": "NumberLiteral", "value": 2 },
    "right": { "type": "NumberLiteral", "value": 3 }
  }
}
```

This corresponds to the intended meaning:

$$1 + (2 * 3)$$

The grammar and recursive descent structure enforce this.

Error Handling in Recursive Descent

Basic approach in this example:

- `expect(type, message)` throws if the next token is not the expected type.
- Each parse function throws helpful messages when it sees something invalid.

More sophisticated options (for later or advanced courses):

- error recovery: skip tokens until a synchronization point (for example, `;`)
- collect multiple errors before giving up

Even a simple recursive descent parser can give helpful error messages because you control the code.

Bottom-up parsing: shift-reduce idea

Bottom-up parsers:

- build the parse tree from the leaves up toward the root
- most common family: LR parsers (including LALR)
- key mechanism: shift-reduce algorithm

Shift-reduce algorithm (very high level):

- maintain:
  - a stack of grammar symbols and parser states
  - the remaining input tokens
- two main actions:
  - shift: move the next input token onto the stack
  - reduce: when the top of the stack matches the right-hand side of a production (a handle), replace it by the left-hand side nonterminal

Compared to LL / recursive descent:

- LR parsers are more powerful (can handle a larger class of grammars)
- but the implementation is more complex and usually table-driven
- so we normally let tools (YACC, Bison, etc.) generate them

Shift-reduce example: NUM + NUM * NUM

Use a simplified expression grammar (with precedence already encoded):

```
Expr   ::= Term ("+" Term)*
Term   ::= Factor ("*" Factor)*
Factor ::= NUM
```

Stack	Input	Action
[ ]	NUM + NUM * NUM $	start
[NUM]	+ NUM * NUM $	shift NUM
[Factor]	+ NUM * NUM $	reduce Factor → NUM
[Term]	+ NUM * NUM $	reduce Term → Factor
[Expr]	+ NUM * NUM $	reduce Expr → Term
[Expr,+]	NUM * NUM $	shift '+'
[Expr,+,NUM]	* NUM $	shift NUM
[Expr,+,Factor]	* NUM $	reduce Factor → NUM
[Expr,+,Term]	* NUM $	reduce Term → Factor
[Expr,+,Term,*]	NUM $	shift '*'
[Expr,+,Term,*,NUM]	$	shift NUM
[Expr,+,Term,*,Factor]	$	reduce Factor → NUM
[Expr,+,Term]	$	reduce Term →Term "*" Factor
[Expr]	$	reduce Expr→Expr "+" Term

- we never explicitly call something like `parseExpr`
- instead, the parser watches the stack and reduces handles
- when the stack is a single `Expr` and input is `$`, the parse succeeds

Parser Generators: ANTLR and Friends

Hand-written recursive descent:

- good for small languages, domain-specific languages, and teaching
- you control every detail of the implementation

Parser generators (ANTLR, JavaCC, YACC/Bison, and others):

- input: grammar file (usually close to BNF or EBNF)
- output: parser code in Java, C++, Python, and other languages
- often include lexer plus parser plus tree-building support

In industry:

- many compilers and tools use parser generators
- some use hand-written recursive descent for flexibility and better error messages

A tiny ANTLR grammar for expressions

```
grammar ExprLang;
expr
   : term (('+' | '-') term)*
   ;
term
   : factor (('*' | '/') factor)*
   ;
factor
   : NUMBER
   | '(' expr ')'
   ;
NUMBER : [0-9]+ ; // Lexer rules (tokens)
WS     : [ \t\r\n]+ -> skip ;
```

- Grammar rules `expr`, `term`, `factor` look almost identical to our handwritten CFG.
- ANTLR generates a lexer and parser from this file.
- Instead of writing `parseExpr`, `parseTerm` by hand, we describe the grammar and let the tool produce the parser code.

Summary of Week 3

- Grammars (CFGs) are the formal tool for describing syntax.
- BNF and EBNF are common notations for writing grammars.
- Parse trees capture how strings are derived from the grammar.
- ASTs are simplified, semantics-focused trees that compilers actually use.
- Recursive descent parsing:
  - one function per nonterminal
  - straightforward to implement in JavaScript
  - gives a working expression parser and AST builder
- Real-world grammars (Java, C++) and tools such as ANTLR scale these ideas up.